TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesor:

Ing. ING. ZAMBRANO CEDENO JORGE ARUN, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

PARALELO: **BARRETO PINCAY HÉCTOR**

SEMESTRE: **QUINTO.** **PARALELO: A**

ACTIVIDAD: **ACTIVIDAD #1: ESTUDIO DE CASO - SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL.**

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad UNO

Actividad # UNO ***DESARROLLO DE LA ACTIVIDAD PROPUESTA.***

DISEÑAR ARQUITECTURA LÓGICA DE UN SISTEMA DE GESTIÓN DE RESCATE Y ADOPCIÓN DE ANIMALES QUE PERMITA APLICAR CORRECTAMENTE PATRONES DE DISEÑO DE SOFTWARE Y FLUFOS DE TRABAJOS PROFESIONALES.

1. MODELO ARQUITECTÓNICO: CLEAN ARCHITECTURE + DDD.

El sistema se dividirá en **4 capas concéntricas**. La regla de oro es que las capas externas conocen a las internas, pero el núcleo (dominio) no conoce a nadie más:

- **Presentación:** Puerta de entrada API REST/GraphQL, Controladores, DTOs.
- **Aplicación:** Casos de uso puros ej. Registrar_Rescate, Procesar_Adopcion.
- **Dominio:** El núcleo del negocio. Contiene las Entidades (Animal), Objetos de Valor (EstadoSalud) y Reglas de Negocio.
- **Infraestructura:** Herramientas y tecnologías externas (Bases de datos, Proveedores de Email/SMS, Pasarelas de Pago).

2. CATÁLOGO DE PATRONES DE DISEÑO A IMPLEMENTAR.

- **State (Comportamiento):** Controlará el ciclo de vida del Animal (Rescatado ➔ Adoptado) y de la Solicitud (Pendiente ➔ Aprobada), eliminando condicionales if/else masivos.
- **Observer (Comportamiento):** Activará acciones automáticas mediante Eventos de Dominio (ej. si un animal pasa a "Listo para adopción", se notifica a la web y a los adoptantes).
- **Adapter (Estructural):** Desacoplará los servicios externos (Stripe, SendGrid, Twilio) de la lógica del sistema, permitiendo cambiarlos en el futuro sin romper nada.
- **Factory Method (Creacional):** Automatizará la creación de perfiles de animales según su especie o tipo de rescate con configuraciones iniciales por defecto.
- **Strategy (Comportamiento):** Permitirá alternar el algoritmo o criterios de evaluación (*scoring*) de los adoptantes según el caso.

3. FLUJO DE TRABAJO PROFESIONAL (WORKFLOW)

- Se utilizará **CQRS** (Separación de Comandos para escribir/modificar datos y Consultas para leerlos).
- Un flujo típico sigue esta secuencia: **Petición (API) ➔ Validación ➔ Manejador de Caso de Uso ➔ Persistencia en Base de Datos ➔ Disparo de Evento de Dominio** (asíncrono para tareas secundarias como correos o asignación de tareas a voluntarios).

DESCRIPCIÓN DE LA ACTIVIDAD.

El estudiante deberá diseñar un sistema orientado a la gestión de casos en un refugio animal. El sistema debe permitir reportar ingresos de animales rescatados (perros, gatos, fauna silvestre), asignar voluntarios o veterinarios encargados y gestionar estados del proceso (ej. Rescatado, En Tratamiento, Adopción).
Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en

GitHub bajo el modelo Gitflow y, aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Para diseñar este sistema de forma profesional, se estructuran **tres componentes clave** que resuelven directamente cada requerimiento operativo del refugio:

1. EL REPORTE DE INGRESO (ENTIDAD CENTRAL).

Se crea un módulo de **Registro Único de Casos**. Cada animal que ingresa genera un expediente digital con un identificador único. Este expediente captura los datos de procedencia (lugar y fecha del rescate), datos biológicos básicos (especie, edad estimada) y el estado crítico inicial de llegada.

2. LA ASIGNACIÓN DE RESPONSABLES (ROLES VINCULADOS).

El sistema implementa un mecanismo de **Asociación Dinámica de Personal**. Al crearse el caso, la plataforma despliega la disponibilidad de los usuarios registrados bajo dos roles específicos: **Veterinarios** (para el flujo clínico) y **Voluntarios** (para el flujo de logística y cuidados). El sistema vincula el ID del responsable directamente al expediente del animal para asegurar la trazabilidad del caso.

3. LA GESTIÓN DE ESTADOS (FLUJO DE PROCESOS).

Se define una **Máquina de Estados Secuencial**. El caso avanza estrictamente a través de fases validadas por el sistema, donde cada estado tiene sus propias reglas:

- **Estado 1: Rescatado.** Fase inicial de recepción y aislamiento preventivo.
- **Estado 2: En Tratamiento.** Se activa automáticamente en cuanto el veterinario asignado prescribe un protocolo médico. El animal no es elegible para salir del refugio.
- **Estado 3: En Adopción.** Se desbloquea únicamente cuando el área médica firma el alta, habilitando el perfil del animal en el catálogo público.

RESUMEN-REPASO EXPLÍCITO (DISEÑO CONCEPTUAL).

- **Ingresos:** Expediente digital único con datos biológicos y de rescate.
- **Asignación:** Vinculación directa de IDs de Veterinarios o Voluntarios al caso.
- **Estados:** Flujo rígido y validado de tres fases (Rescatado ➔ En Tratamiento ➔ En Adopción).

INSTRUCCIONES.

ANÁLISIS DEL PROBLEMA.

1. Identificar los actores principales.

Ciudadano/Reportante, Voluntario/Veterinario, Administrador del Refugio.

2. Definir los requerimientos funcionales.

Crear caso de rescate, categorizar especie, asignar voluntario y,

3. Definir requerimientos no funcionales.

Escalabilidad, mantenibilidad, trazabilidad de versiones).

1. ACTORES PRINCIPALES DEL SISTEMA.

Definen quiénes interactúan con la plataforma y cuáles son sus fronteras de responsabilidad:

- **Ciudadano / Reportante:** Actor externo que inicia el flujo. Su única función es notificar un animal en situación de riesgo, aportando ubicación, fotos y estado visual inicial. No tiene acceso al panel interno.
- **Voluntario:** Actor interno operativo. Se encarga de la logística de rescate en campo, el traslado del animal y los cuidados diarios en el refugio (alimentación, paseos).
- **Veterinario:** Actor interno especialista. Responsable exclusivo del diagnóstico clínico, prescripción de tratamientos médicos, cirugías y emisión de altas médicas para adopción.
- **Administrador del Refugio:** Actor interno de control. Posee control total sobre el sistema. Gestiona el catálogo de usuarios, asigna cargas de trabajo, audita el historial de los casos y genera reportes estadísticos.

2. REQUERIMIENTOS FUNCIONALES (RF).

Definen lo que el sistema **debe hacer** explícitamente:

- **RF-01: Crear Caso de Rescate:** El sistema debe permitir registrar un nuevo caso (ya sea por reporte ciudadano o ingreso directo), generando un código de seguimiento único e inmutable.
- **RF-02: Categorizar Especie y Taxonomía:** El sistema debe clasificar obligatoriamente al animal al momento del ingreso en categorías predefinidas (Canino, Felino, Ave, Fauna Silvestre, etc.) para aplicar el protocolo logístico y legal correspondiente.
- **RF-03: Asignar responsable (Voluntario/Veterinario):** El sistema debe permitir al Administrador vincular un caso de rescate a uno o varios voluntarios para la captura/traslado, y a un veterinario específico para su evaluación clínica.

3. REQUERIMIENTOS NO FUNCIONALES (RNF).

Definen los atributos de calidad y restricciones bajo las que **debe operar** el sistema:

- **RNF-01: Escalabilidad:** La arquitectura lógica debe estar desacoplada (Capas/DDD) para permitir el crecimiento del sistema (ej. pasar de un monolito a microservicios) si el volumen de animales rescatados o refugios integrados aumenta exponencialmente.
- **RNF-02: Mantenibilidad:** El código debe seguir principios SOLID y patrones de diseño claros. Esto garantiza que añadir una nueva especie o un nuevo estado en el flujo de trabajo requiera modificaciones mínimas sin romper el sistema existente.
- **RNF-03: Trazabilidad de Versiones (Auditoría e Historial):** El sistema debe registrar un historial detallado e inmutable (*logs* de auditoría) de cada cambio de estado, asignación de personal o modificación médica en el expediente del animal, detallando fecha, hora y el actor que realizó la acción.

RESUMEN-REPASO EXPLÍCITO (ANÁLISIS DEL PROBLEMA).

- **ACTORES:**
  Ciudadano (Reporta), Voluntario (Opera), Veterinario (Cura) y Administrador (Controla).
- **REQUERIMIENTOS FUNCIONALES:**
  Registro único de casos, clasificación taxonómica obligatoria y asignación dirigida de personal.

- **REQUERIMIENTOS NO FUNCIONALES:**
  Escalabilidad arquitectónica, mantenibilidad basada en patrones y trazabilidad estricta (auditoría de cambios).

APLICACIÓN DE PATRONES DE DISEÑO.

PATRÓN DE DISEÑO "FACTORY METHOD" (MÉTODO DE FÁBRICA).

El Factory Method (en español, Método de Fábrica) es simplemente una técnica de programación (un molde o plantilla) que sirve para organizar la creación de objetos sin enredar el código con demasiados condicionales if o else.

¿QUÉ PROBLEMA RESUELVE?

En un refugio, un perro, un gato y un animal de fauna silvestre no requieren el mismo tipo de expediente ni los mismos cuidados médicos iniciales. Si utilizáramos un código lleno de condiciones if/else para crear cada tipo de ficha, el sistema sería difícil de mantener.

¿CÓMO SE APLICA EN EL SISTEMA DE RESCATE?

Implementamos una clase creadora abstracta (ExpedienteFactory) que define el método para crear el expediente, y clases hijas concretas que devuelven el objeto específico según la especie:

- **Perro:** Requiere control de vacunas caninas y chip de identificación.
- **Gato:** Requiere pruebas felinas y manejo específico.
- **Fauna Silvestre:** Requiere protocolo de rehabilitación con el Ministerio del Ambiente y control de cuarentena estricta.

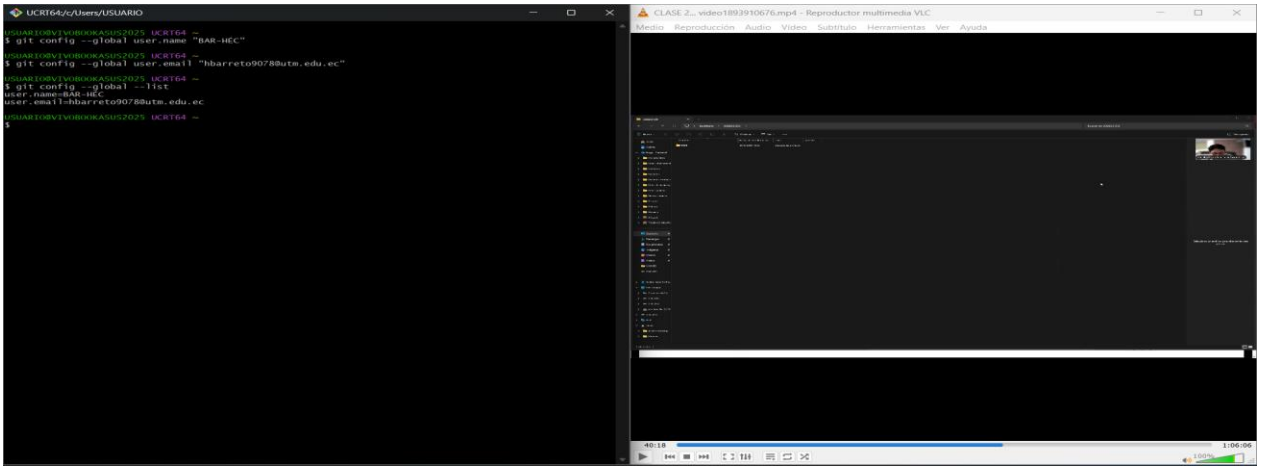
RESUMEN DE PATRONES DE DISEÑO.

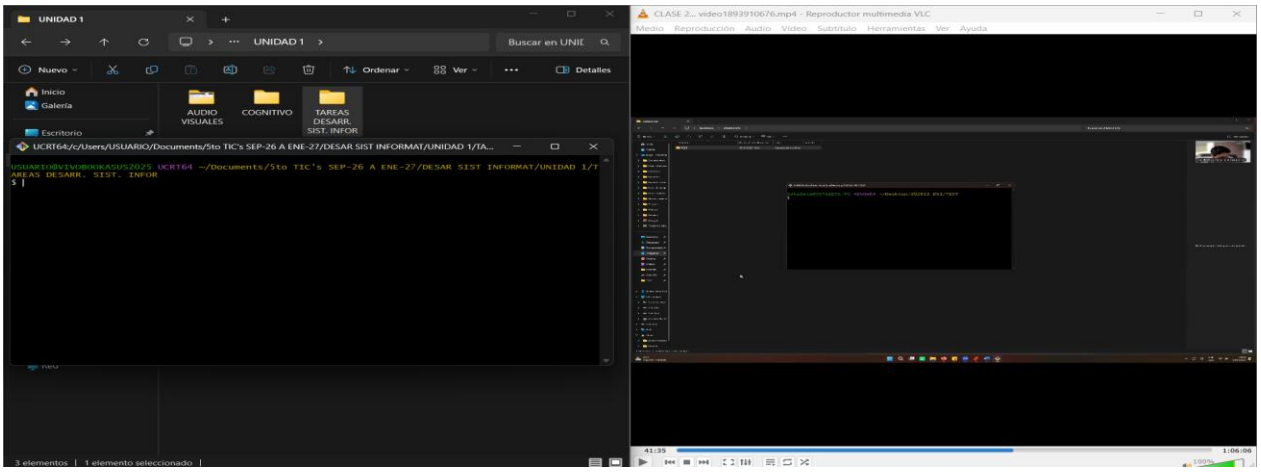
PATRÓN DE DISEÑO	PROPÓSITO EN EL SISTEMA DE RESCATE
1. Singleton	Garantiza que exista una única instancia de conexión a la base de datos central y gestión de configuración del refugio.
2. Observer	Notifica automáticamente vía correo o alerta móvil a los voluntarios disponibles cuando se registra un nuevo caso urgente.
3. Factory Method *(Enfatizado)*	Centraliza y desacopla la creación de los expedientes según la especie (Perro, Gato, Silvestre), facilitando futuras ampliaciones.

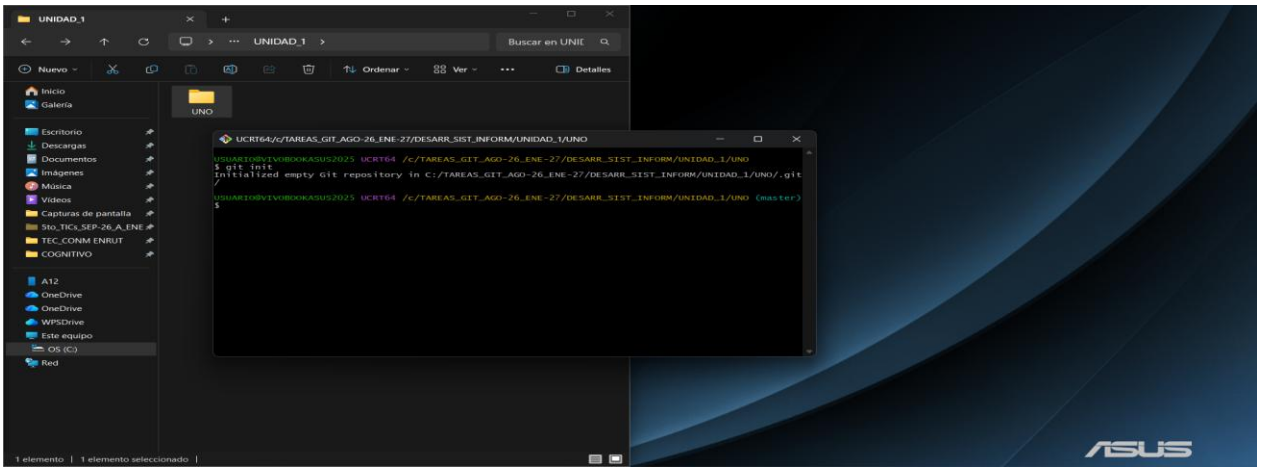
DISEÑO DEL SISTEMA.

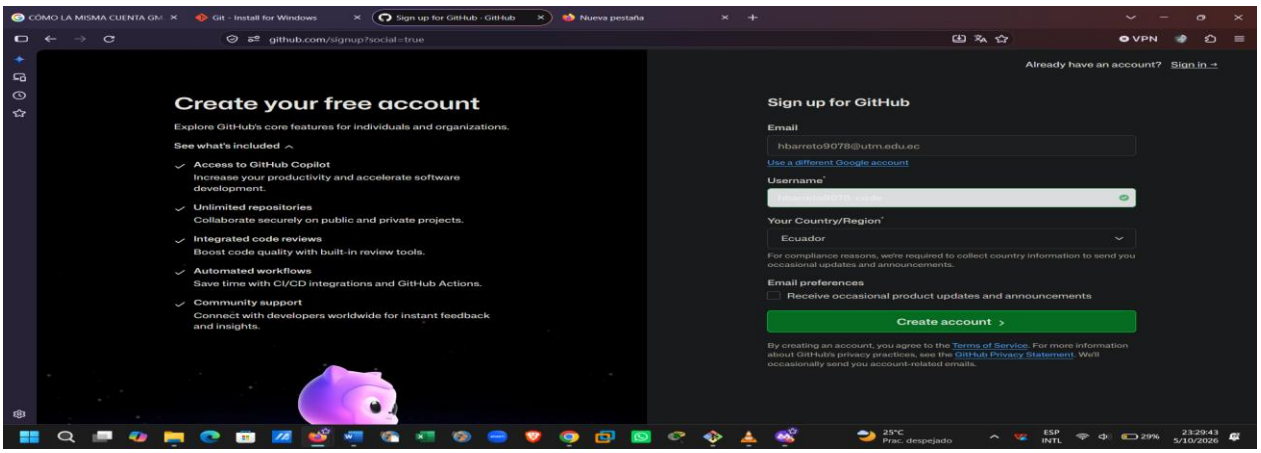
- **Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).**

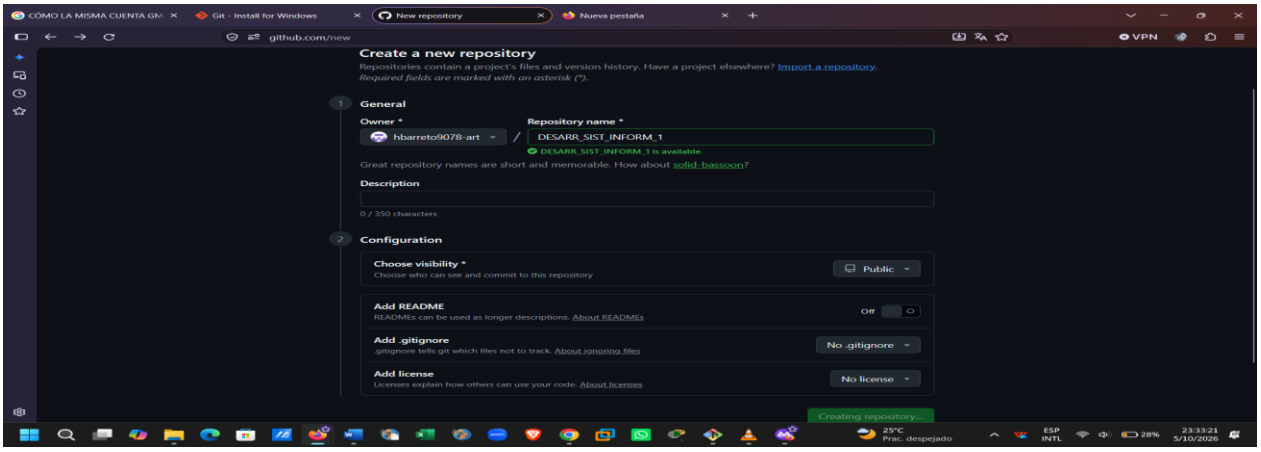
Se adjunta imagen que evidencian de creación de repositorio GITHUB que configuran estructura de ramas gitflow.

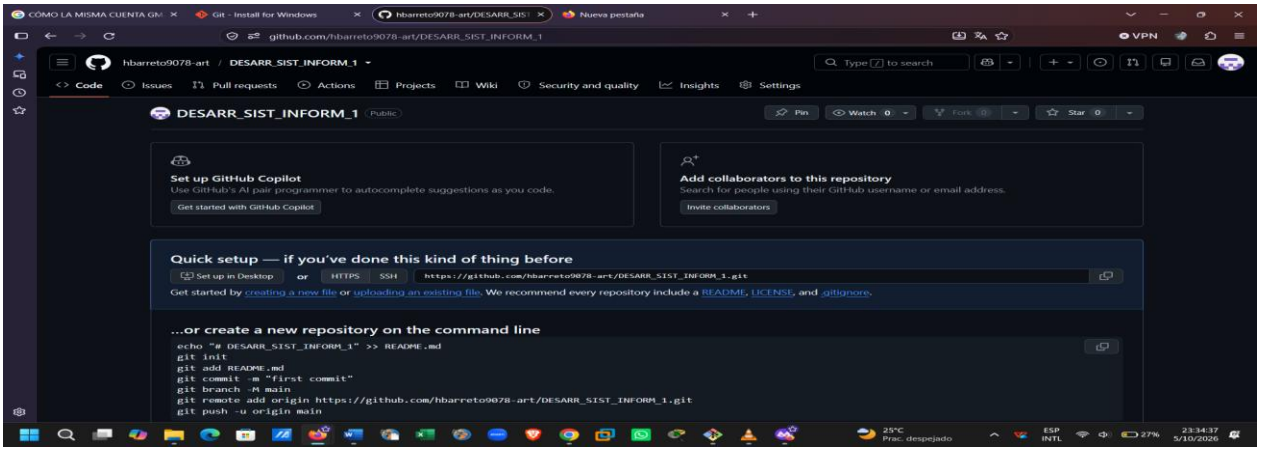


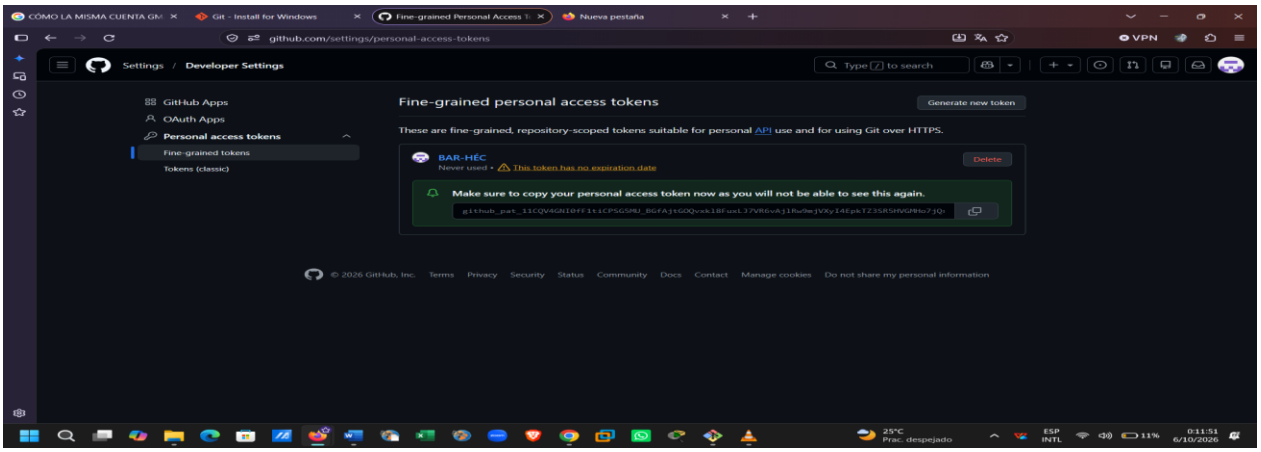


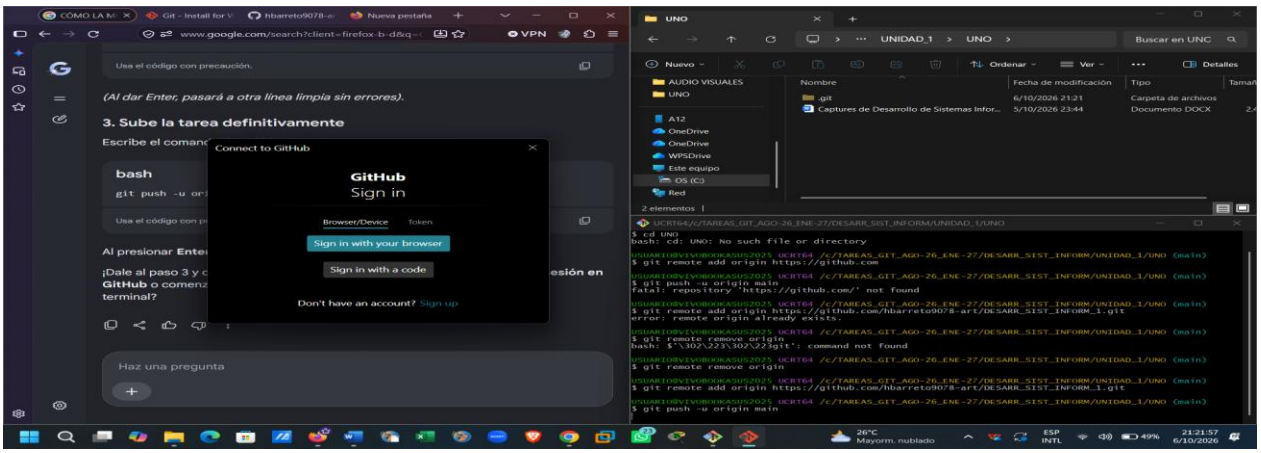


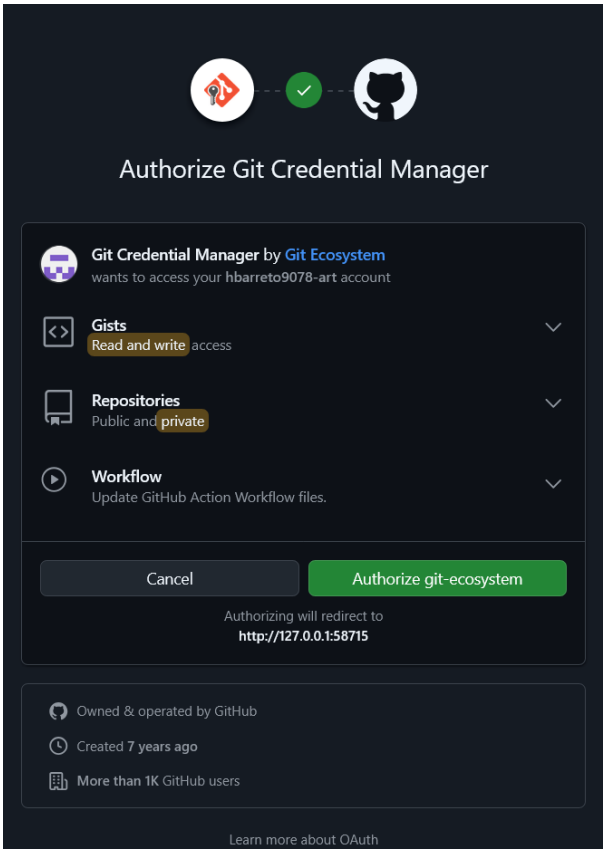

Create your free account
Explore GitHub's core features for individuals and organizations.
See what's included
Access to GitHub Copilot
Increase your productivity and accelerate software development.
Unlimited repositories
Collaborate securely on public and private projects.
Integrated code reviews
Boost code quality with built-in review tools.
Automated workflows
Save time with CI/CD integrations and GitHub Actions.
Community support
Connect with developers worldwide for instant feedback and insights.
Already have an account? Sign in
Sign up for GitHub
Email
hbarreto9078@utm.edu.ec
Use a different Google account
Username
Your Country/Region
Ecuador
Email preferences
Receive occasional product updates and announcements
Create account

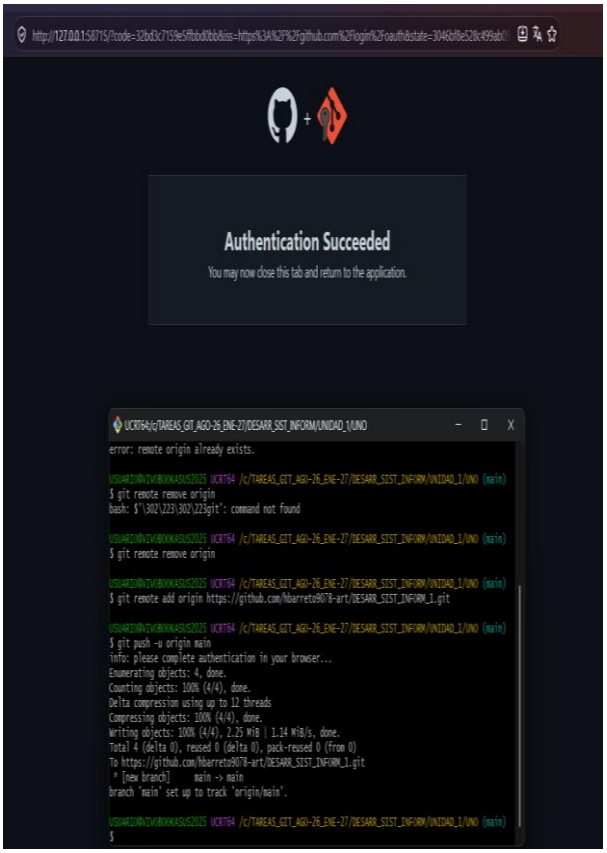

Create a new repository
Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
General
Owner *
hbarreto9078-art
Repository name *
DESARR_SIST_INFORM_1
DESARR_SIST_INFORM_1 is available.
Great repository names are short and memorable. How about solid-bassoon?
Description
Configuration
Choose visibility *
Public
Add README
Off
Add .gitignore
No .gitignore
Add license
No license
Creating repository...


hbarreto9078-art / DESARR_SIST_INFORM_1
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
DESARR_SIST_INFORM_1
Public
Set up GitHub Copilot
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
Get started with GitHub Copilot
Add collaborators to this repository
Search for people using their GitHub username or email address.
Invite collaborators
Quick setup — if you've done this kind of thing before
https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
Get started by creating a new file or uploading an existing file. We recommend every repository include a README, LICENSE, and .gitignore.
...or create a new repository on the command line
echo "# DESARR_SIST_INFORM_1" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
git push -u origin main


Settings / Developer Settings
GitHub Apps
OAuth Apps
Personal access tokens
Fine-grained tokens
Tokens (classic)
Fine-grained personal access tokens
Generate new token
These are fine-grained, repository-scoped tokens suitable for personal API use and for using Git over HTTPS.
BAR-HÉC
Never used
This token has no expiration date
Delete
Make sure to copy your personal access token now as you will not be able to see this again.


3. Sube la tarea definitivamente
Connect to GitHub
GitHub
Sign in
Browser/Device
Token
Sign in with your browser
Sign in with a code
Don't have an account? Sign up
Haz una pregunta
UNO
UNIDAD_1 > UNO
AUDIO VISUALES
UNO
Captures de Desarrollo de Sistemas Infor...
6/10/2026 21:21
5/10/2026 23:44
$ git remote add origin https://github.com
fatal: repository 'https://github.com/' not found
error: remote origin already exists.
$ git remote add origin https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
bash: $'\302\223\302\223git': command not found
$ git remote add origin https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
$ git push -u origin main


Authorize Git Credential Manager
Git Credential Manager by Git Ecosystem
wants to access your hbarreto9078-art account
Gists
Read and write access
Repositories
Public and private
Workflow
Update GitHub Action Workflow files.
Cancel
Authorize git-ecosystem
Authorizing will redirect to
http://127.0.0.1:58715
Owned & operated by GitHub
Created 7 years ago
More than 1K GitHub users
Learn more about OAuth


Authentication Succeeded
You may now close this tab and return to the application.
error: remote origin already exists.
$ git remote remove origin
bash: $'\302\223\302\223git': command not found
$ git remote remove origin
$ git remote add origin https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.25 MiB | 1.14 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1.git
* [new branch] main -> main
branch 'main' set up to track 'origin/main'.

CREAR UN DIAGRAMA DE CASOS DE USO PARA ENTENDER LAS INTERACCIONES DEL PERSONAL CON LA PLATAFORMA.

DIAGRAMA DE CASOS DE USO (ESTRUCTURA UML).

1. ACTORES DEL SISTEMA.

- **Ciudadano (Reportante)**

  [Actor Externo]

- **Voluntario** [Actor Interno]
- **Veterinario** [Actor Interno]
- **Administrador**

  Actor Interno / Superusuario

2 **"PLATAFORMA DE GESTIÓN DEL REFUGIO"**

SISTEMA DE GESTIÓN DE CASOS (REFUGIO)

Ciudadano	CU-01: Reportar Animal en Riesgo
Voluntario	CU-02: Gestionar Logística de Traslado. CU-03: Actualizar Bitácora de Cuidados Diarios
Veterinario	CU-04: Registrar Evaluación Clínica CU-05: Prescribir Tratamiento Médico
Administrador	CU-06: Crear Caso de Rescate Directo CU-07: Asignar Responsable a Caso CU-08: Cambiar Estado del Proceso)

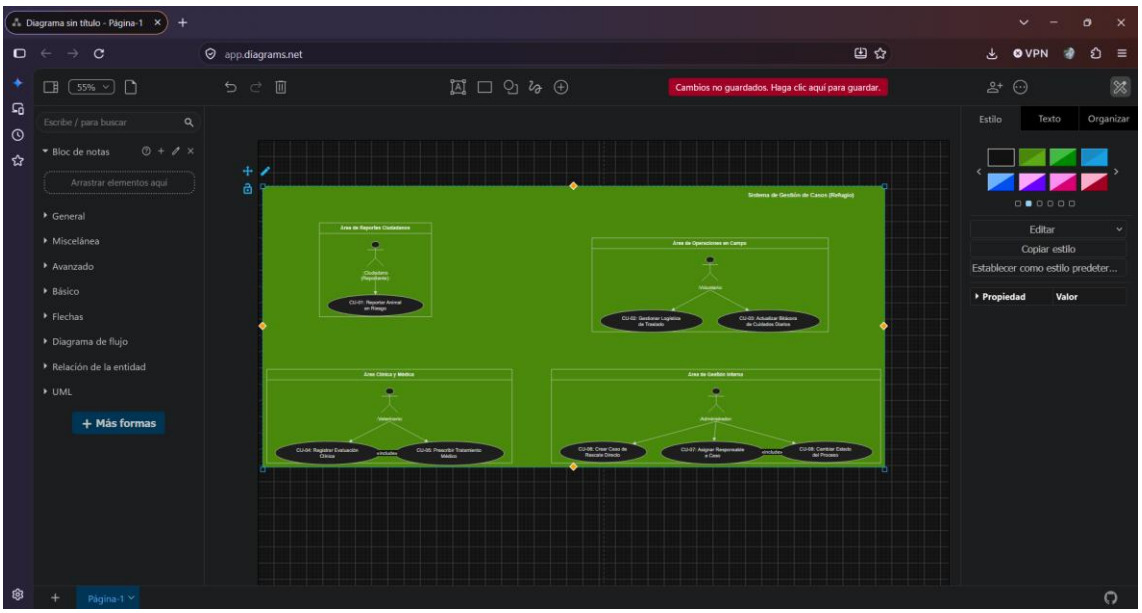
3. RELACIONES DE INCLUSIÓN <<INCLUDE>> Y, EXTENSIÓN <<EXTEND>>

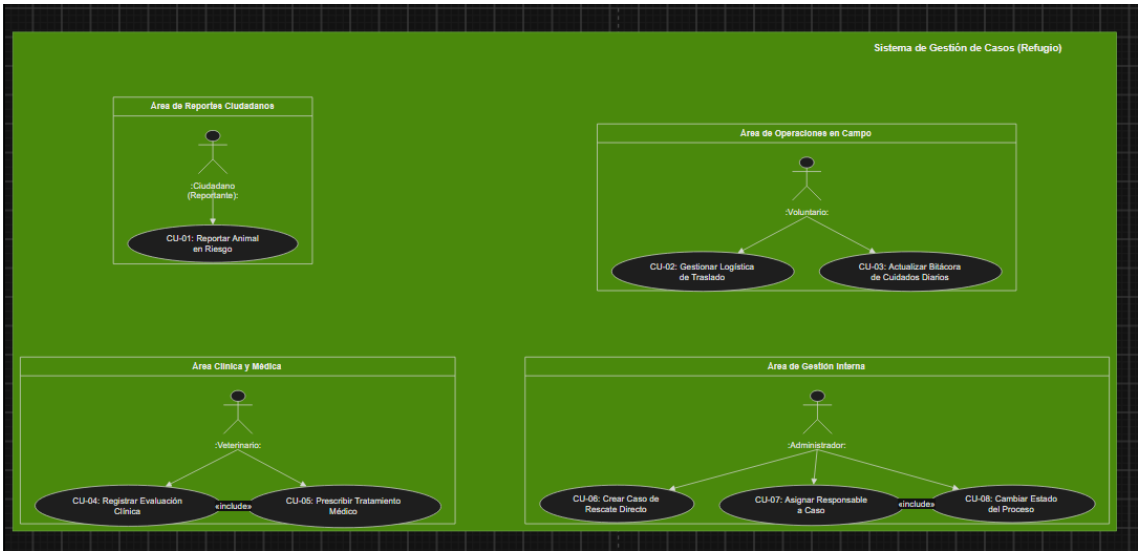
Para que el sistema mantenga una lógica profesional, ciertos casos de uso dependen obligatoriamente de otros:

- **(CU-07: Asignar responsable a caso)** <<include>> **(CU-08: Cambiar Estado del Proceso)**
  - *Explicación:* El Administrador no puede mover un caso al estado "En Tratamiento" si el sistema no incluye primero la asignación obligatoria de un Veterinario responsable.
- **(CU-05: Prescribir Tratamiento Médico)** <<include>> **(CU-04: Registrar Evaluación Clínica)**
  - *Explicación:* Un Veterinario no puede recetar medicamentos en la plataforma sin que el caso incluya un registro previo de su diagnóstico clínico.

RESUMEN-REPASO EXPLÍCITO (DIAGRAMA DE CASOS DE USO).

- **FRONTERA:** Separa las acciones externas del ciudadano de las operaciones internas del personal del refugio.
- **INTERACCIONES CLAVE:** El Ciudadano solo reporta; el Voluntario opera en campo; el Veterinario gestiona la salud; el Administrador orquesta asignaciones y estados.
- **REGLA DE NEGOCIO EN UML:** Se aplican relaciones <<include>> para garantizar que ninguna transición de estado ocurra sin un responsable asignado o un diagnóstico previo.


Diagrama sin título - Página-1
app.diagrams.net
Cambios no guardados. Haga clic aquí para guardar.
Escribe / para buscar
Bloc de notas
Arrastrar elementos aquí
General
Miscelánea
Avanzado
Básico
Flechas
Diagrama de flujo
Relación de la entidad
UML
+ Más formas
Estilo
Texto
Organizar
Editar
Copiar estilo
Establecer como estilo predeter...
Propiedad
Valor
Página-1


Sistema de Gestión de Casos (Refugio)
Área de Reportes Ciudadanos
:Ciudadano (Reportante):
CU-01: Reportar Animal en Riesgo
Área de Operaciones en Campo
:Voluntario:
CU-02: Gestionar Logística de Traslado
CU-03: Actualizar Bitácora de Cuidados Diarios
Área Clínica y Médica
:Veterinario:
CU-04: Registrar Evaluación Clínica
«include»
CU-05: Prescribir Tratamiento Médico
Área de Gestión Interna
:Administrador:
CU-06: Crear Caso de Rescate Directo
CU-07: Asignar Responsable a Caso
«include»
CU-08: Cambiar Estado del Proceso

Evidencias

(*Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).*

Link al repositorio github…
https://github.com/hbarreto9078-art/DESARR_SIST_INFORM_1

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)

1. **Refactoring.Guru**. Catálogo detallado de patrones de diseño de software, incluyendo explicaciones paso a paso de **Factory Method**, **Singleton** y **Observer** con diagramas UML. Enlace directo a Refactoring.Guru
2. **Atlassian Gitflow Workflow Documentation**. Guía oficial sobre la gestión de ramas con el modelo Gitflow (master, develop y feature branches). Enlace directo a Atlassian Gitflow
3. **Atlassian Gitflow Workflow Documentation**. Guía oficial sobre la gestión de ramas con el modelo Gitflow (master, develop y feature branches). Enlace directo a Atlassian Gitflow *https://dl.acm.org/doi/epdf/10.1145/355604.361591*
4. *https://www.researchgate.net/publication/334946581_Utilizing_Software_Design_Patterns_in_Product-Driven_Manufacturing_System_A_Case_Study*